Compiler Design

Detailed Semester Exam Notes | Theory, Diagrams, Tables and Concepts

Prepared for Rahul Yadav. These notes cover compiler phases, lexical analysis, parsing, syntax directed translation, type checking, runtime environment, code generation and optimization according to the provided syllabus.

Unit	Syllabus Area	Main Exam Focus
I	Introduction to compiling and lexical analysis	Compiler phases, data structures, tokens, input buffering, LEX
II	Syntax analysis and syntax directed translation	CFG, parsing methods, LR parsers, parse trees, attributes
III	Type checking and runtime environment	Type systems, conversions, activation records, storage allocation
IV	Code generation	Intermediate code, backpatching, basic blocks, DAG, register allocation
V	Code optimization	Local/global optimization, data flow, dead code, loop optimization

Exam tip: for every long answer, write definition, diagram, working steps, example and advantages/limitations. Compiler Design answers become strong when you include small grammar, three-address code or flow graph examples.

UNIT I: Introduction to Compiler

A compiler is a system software that translates a program written in a high-level language into an equivalent target program, usually machine code, assembly code or intermediate code. During translation, it also detects errors and reports them to the programmer.

The input program is called source program and the output is called target program. A compiler is different from an interpreter because a compiler translates the whole program before execution, whereas an interpreter translates and executes statement by statement.

The compiler must preserve the meaning of the source program while producing efficient target code. This requires analysis of the program structure and synthesis of the target code.

- Compiler improves execution speed because translated code can run directly.
- It reports lexical, syntax, semantic and sometimes optimization-related errors.
- It uses symbol table to store information about identifiers.
- It may generate intermediate code before final target code.

Term	Meaning
Source Program	Input high-level language program
Target Program	Generated low-level or intermediate program
Translator	Software that converts one language to another
Error Handler	Reports invalid program constructs
Symbol Table	Stores identifier attributes

Types of Compiler and Major Compiler Data Structures

Compilers can be classified according to translation method, target platform and execution environment. A single-pass compiler scans source program once, while a multi-pass compiler scans it multiple times. A cross compiler runs on one machine but generates code for another machine.

Important data structures used in compiler include symbol table, syntax tree, parse tree, literal table, intermediate code list, control flow graph and DAG. These structures help store program information between phases.

The symbol table is one of the most important structures. It stores name, type, scope, size, memory location and other attributes of identifiers.

- Single-pass compiler is fast but less flexible.
- Multi-pass compiler allows better analysis and optimization.
- Cross compiler is used in embedded system development.
- Just-in-time compiler translates code during execution.
- Compiler data structures reduce repeated analysis.

Compiler Type	Meaning	Example/Use
Single Pass	Processes source once	Simple languages
Multi Pass	Uses multiple scans	Optimizing compilers
Cross Compiler	Host and target differ	Embedded systems
JIT Compiler	Compiles at run time	JVM/.NET
Source-to-Source	High-level to high-level	TypeScript to JavaScript

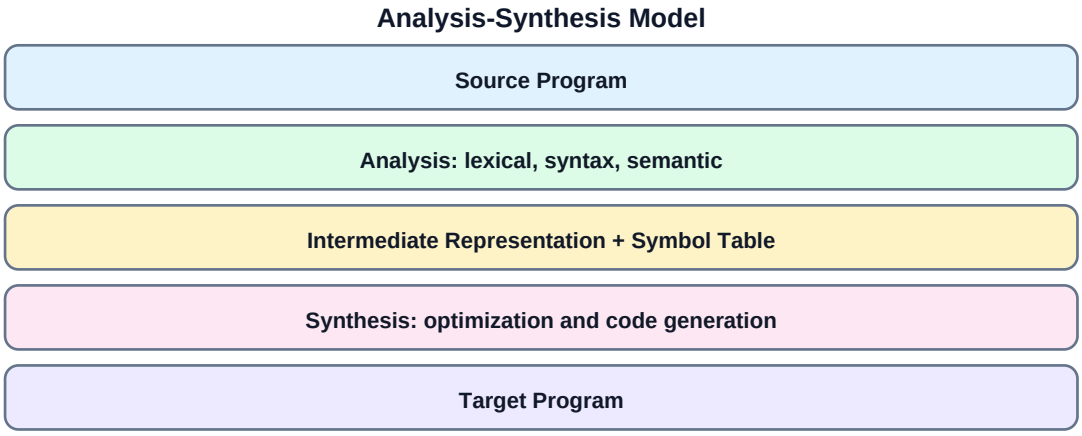
Front End, Back End and Analysis-Synthesis Model

The compiler can be divided into front end and back end. The front end is machine independent and analyzes the source program. It performs lexical analysis, syntax analysis, semantic analysis and intermediate code generation.

The back end is machine dependent and generates optimized target code. It performs machine-dependent code optimization, instruction selection, register allocation and final code generation.

The analysis-synthesis model states that compilation has two major parts. Analysis breaks the source program into meaningful parts and checks correctness. Synthesis constructs the target program from analyzed information.

- Front end depends mainly on source language.
- Back end depends mainly on target machine.
- Intermediate representation connects front end and back end.
- Analysis creates symbol table and intermediate form.
- Synthesis generates optimized target code.



Part	Main Work	Machine Dependency
Front End	Analyze source program and generate IR	Mostly machine independent
Middle End	Optimize intermediate code	Mostly machine independent
Back End	Generate target code	Machine dependent

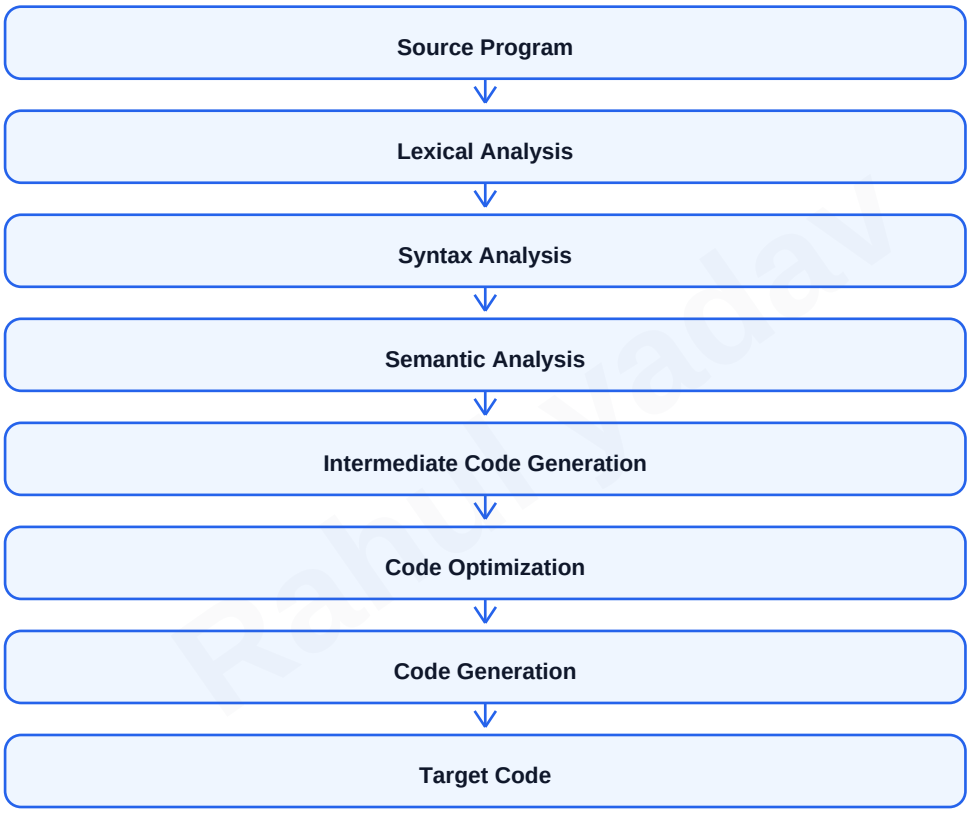
Phases of Compiler

A compiler is organized into phases. Each phase takes input from previous phase and produces output for next phase. This modular organization makes compiler design easier and helps error detection at different levels.

The major phases are lexical analysis, syntax analysis, semantic analysis, intermediate code generation, code optimization and target code generation. Symbol table management and error handling support all phases.

For exam answers, draw the full phase diagram and explain the output of each phase with a small expression such as position = initial + rate * 60.

- Lexical analyzer converts characters into tokens.
- Syntax analyzer checks grammar and builds parse tree.
- Semantic analyzer checks meaning and types.
- Intermediate code generator produces machine-independent code.
- Optimizer improves code efficiency.
- Code generator produces target code.



Phase	Input	Output
Lexical Analysis	Characters	Tokens
Syntax Analysis	Tokens	Parse tree/syntax tree
Semantic Analysis	Syntax tree	Annotated tree
ICG	Annotated tree	Intermediate code
Optimization	Intermediate code	Improved intermediate code
Code Generation	Optimized IR	Target code

Lexical Analysis and Tokens

Lexical analysis is the first phase of compiler. It reads the source program as a stream of characters and groups them into meaningful units called tokens. It removes white spaces, comments and unnecessary characters.

A token is a pair consisting of token name and optional attribute value. For example, in count = count + 1, identifiers, assignment operator, plus operator and number are tokens. The actual character sequence matched for a token is called lexeme.

The lexical analyzer also enters identifiers into the symbol table and reports lexical errors such as illegal characters or unterminated strings.

- Token is a category such as ID, NUM, KEYWORD or OPERATOR.
- Lexeme is actual string from source program.
- Pattern is rule that describes valid lexemes.
- Regular expressions are used to specify token patterns.
- Finite automata are used to recognize tokens.

Concept	Meaning	Example
Token	Class/category of lexeme	ID
Lexeme	Actual character sequence	total
Pattern	Rule for token	letter(letter|digit)*
Attribute	Extra information	symbol table pointer
Keyword	Reserved word	if, while, int

```
Source: if (x >= 10) x = x + 1;
Tokens: IF, LPAREN, ID(x), RELOP(>=), NUM(10),
        RPAREN, ID(x), ASSIGN, ID(x), PLUS, NUM(1), SEMI
```

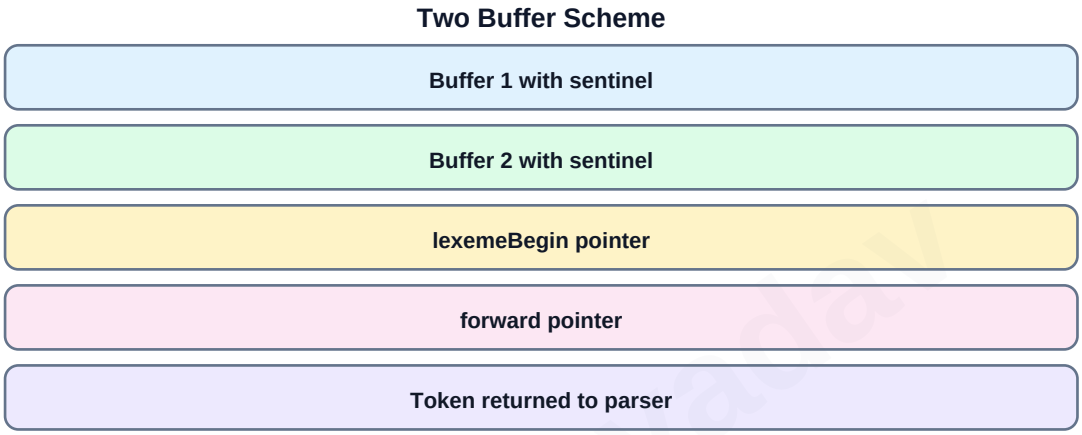
Input Buffering in Lexical Analyzer

Input buffering improves lexical analyzer performance. Reading one character at a time from disk is slow, so the input is read in blocks. The scanner then processes characters from memory buffer.

A common technique is two-buffer scheme. Two equal-size buffers are used with two pointers: lexemeBegin and forward. lexemeBegin marks the start of current lexeme and forward scans ahead to find token end.

Sentinel characters are placed at the end of buffers to reduce boundary checks. When forward reaches sentinel, the next buffer is loaded. This improves scanner speed.

- Buffering reduces expensive input operations.
- lexemeBegin marks beginning of lexeme.
- forward pointer scans characters.
- Sentinel helps detect end of buffer.
- Retracting forward may be needed for lookahead.



Pointer	Purpose
lexemeBegin	Points to first character of current lexeme
forward	Scans ahead until token pattern matches
Sentinel	Special marker showing end of buffer
Retract	Move pointer back after extra lookahead

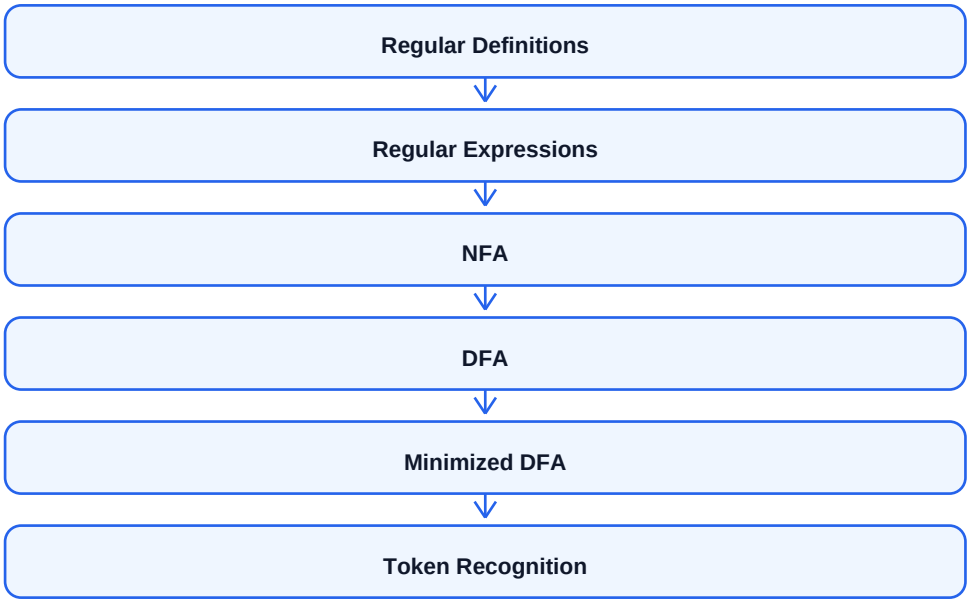
Specification and Recognition of Tokens

Token specification defines valid lexemes using regular expressions. Regular definitions give names to common patterns such as digit, letter and identifier. The lexical analyzer uses these patterns to recognize tokens.

Recognition of tokens can be implemented using finite automata. A regular expression is converted to NFA, then NFA can be converted to DFA. DFA is efficient because for every state and input symbol, next state is uniquely determined.

If multiple token patterns match the same input, the lexical analyzer usually uses longest match rule and priority rule. Longest match selects the longest lexeme; priority resolves ties.

- Regular expression specifies token pattern.
- NFA may have epsilon transitions and multiple choices.
- DFA has exactly one transition for each state-symbol pair.
- Longest match is important for operators like > and >=.
- Keywords are often recognized before identifiers or by symbol table lookup.




```
digit  -> [0-9]
letter -> [A-Za-z]
id     -> letter (letter | digit)*
num    -> digit+
relop  -> < | <= | > | >= | == | !=
```

LEX: Lexical Analyzer Generator

LEX is a tool used to generate lexical analyzers automatically. The programmer writes token patterns and actions in a LEX specification file. LEX converts this specification into C code for scanner function yylex().

A LEX program has three sections: definitions, rules and user subroutines. Definitions contain declarations and regular definitions. Rules contain pattern-action pairs. User subroutines contain helper C functions.

LEX is commonly used with YACC/Bison. LEX returns tokens to parser and parser checks grammatical structure.

- LEX reduces manual scanner writing.
- Rules are written as regular expression followed by action.
- yytext stores matched lexeme.
- yyleng stores lexeme length.
- yylex() is the scanner function generated by LEX.

Section	Purpose
Definitions	C declarations and regular definitions
Rules	Token patterns and actions
User Subroutines	Helper functions and main function
yytext	Matched lexeme
yylex	Scanner function

```
%{
#include <stdio.h>
%}
digit [0-9]
%%
{digit}+   { printf("NUMBER: %s\n", yytext); }
[a-zA-Z]+  { printf("ID: %s\n", yytext); }
[ \t\n]   ;
.          { printf("Invalid character"); }
%%
```

UNIT II: Syntax Analysis and CFG

Syntax analysis is the second phase of compiler. It checks whether the sequence of tokens follows the grammar of the programming language. The syntax analyzer is also called parser.

A context-free grammar is used to specify the syntax of programming languages. A CFG has terminals, non-terminals, production rules and start symbol. Terminals are tokens, non-terminals are syntactic variables and productions define derivations.

Parser output is a parse tree or syntax tree. If token sequence is invalid, parser reports syntax errors and may try recovery.

- CFG is suitable for nested structures like expressions and statements.
- Parser receives tokens from lexical analyzer.
- Parser verifies grammatical structure.
- Parse tree shows complete derivation.
- Syntax tree is compact and removes unnecessary grammar symbols.

CFG Component	Meaning	Example
Terminal	Token/basic symbol	id, +, if
Non-terminal	Syntactic variable	E, T, stmt
Production	Replacement rule	E -> E + T
Start Symbol	Root grammar symbol	program
Derivation	Applying productions	E => E + T

```
E -> E + T | T
T -> T * F | F
F -> ( E ) | id
```

Parse Tree, Ambiguity and Grammar Transformations

A parse tree is a hierarchical tree representation of derivation. The root is start symbol, internal nodes are non-terminals and leaves are terminals. It shows how a string is derived from grammar.

A grammar is ambiguous if a string has more than one parse tree or more than one leftmost/rightmost derivation. Ambiguity is undesirable because it can produce multiple meanings for same program.

Grammar transformations are used to make grammar suitable for parsing. Common transformations are eliminating left recursion and left factoring. Left recursion causes problems in top-down parsing.

- Ambiguous grammar should be rewritten or resolved using precedence and associativity.
- Immediate left recursion has form A -> A alpha | beta.
- After elimination: A -> beta A' and A' -> alpha A' | epsilon.
- Left factoring removes common prefixes.
- Transformed grammar should preserve language meaning.


```
Left recursive:
A -> A alpha | beta

After removal:
A  -> beta A'
A' -> alpha A' | epsilon
```

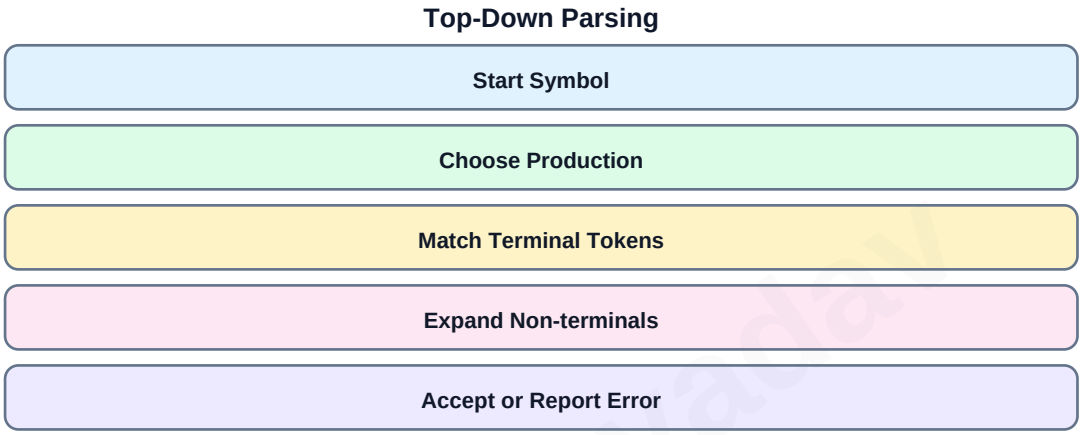
Top-Down Parsing and Recursive Descent Parsing

Top-down parsing constructs parse tree from root to leaves. It starts with start symbol and tries to derive the input string. It uses leftmost derivation.

Recursive descent parsing is a top-down parsing method where each non-terminal has a corresponding procedure. The procedure tries to match productions and consume input tokens.

A simple recursive descent parser may require backtracking. Predictive parsing is a special recursive descent method without backtracking and uses lookahead token to choose production.

- Top-down parser expands non-terminals from root.
- Recursive descent parser is easy to implement manually.
- Backtracking can be inefficient.
- Grammar should be free from left recursion.
- Predictive parser uses FIRST and FOLLOW sets.




```
void E() {
    T();
    Eprime();
}
void Eprime() {
    if (lookahead == '+') {
        match('+'); T(); Eprime();
    }
}
```

Predictive Parsing, FIRST and FOLLOW

Predictive parsing is a non-backtracking top-down parsing technique. It uses a parsing table and one lookahead token to select correct production. LL(1) grammars are suitable for predictive parsing.

FIRST(alpha) is the set of terminals that can appear first in strings derived from alpha. FOLLOW(A) is the set of terminals that can appear immediately to the right of non-terminal A in some sentential form.

The predictive parsing table is constructed using FIRST and FOLLOW sets. If a table cell has more than one production, grammar is not LL(1).

- LL(1) means Left-to-right scan, Leftmost derivation, 1 lookahead.
- FIRST helps choose production based on starting terminal.
- FOLLOW is used when production can derive epsilon.
- Parsing stack initially contains start symbol and end marker.
- No backtracking is needed in predictive parsing.

Set	Definition	Use
FIRST(X)	Terminals that can begin strings from X	Choose production
FOLLOW(A)	Terminals that can follow A	Handle epsilon productions
LL(1) Table	Non-terminal x terminal matrix	Parser decision
Lookahead	Current input token	Select table entry

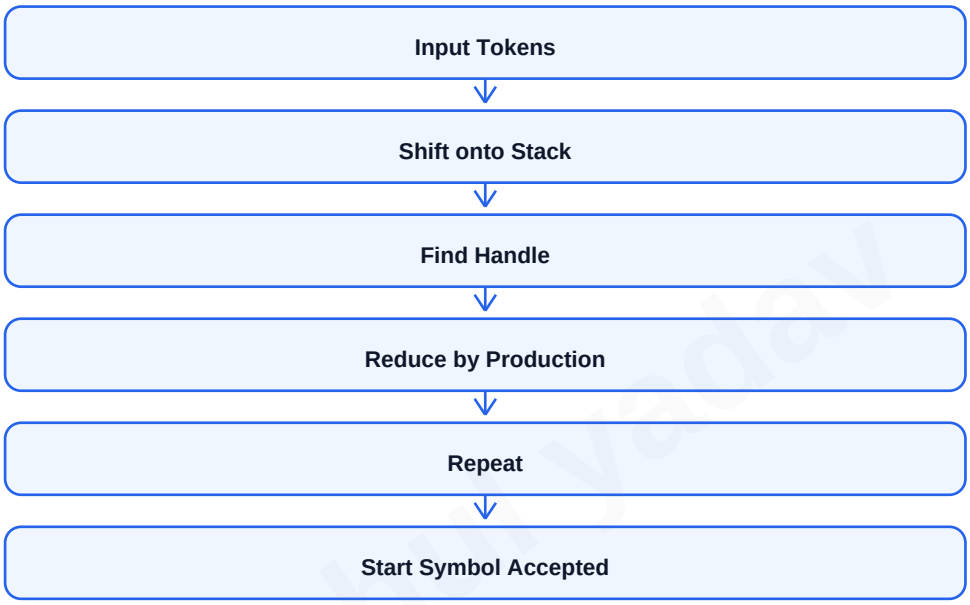
Bottom-Up Parsing and Operator Precedence Parsing

Bottom-up parsing constructs parse tree from leaves to root. It starts from input tokens and reduces substrings to non-terminals until the start symbol is obtained. It corresponds to reverse of rightmost derivation.

Shift-reduce parsing is a common bottom-up method. Shift means push next input symbol onto stack. Reduce means replace handle on stack by a non-terminal using a production.

Operator precedence parsing is used for expression grammars. It uses precedence and associativity relations between operators to decide whether to shift or reduce.

- Bottom-up parser can handle a larger class of grammars than predictive parser.
- A handle is substring that matches right side of a production.
- Shift-reduce conflicts occur when parser cannot decide shift or reduce.
- Operator precedence parser cannot handle all CFGs.
- Precedence table controls expression parsing.



Action	Meaning
Shift	Move next input symbol to stack
Reduce	Replace handle by non-terminal
Accept	Parsing completed successfully
Error	No valid shift/reduce action
Handle	Substring matching RHS of production

LR Parsers: SLR, LALR and Canonical LR

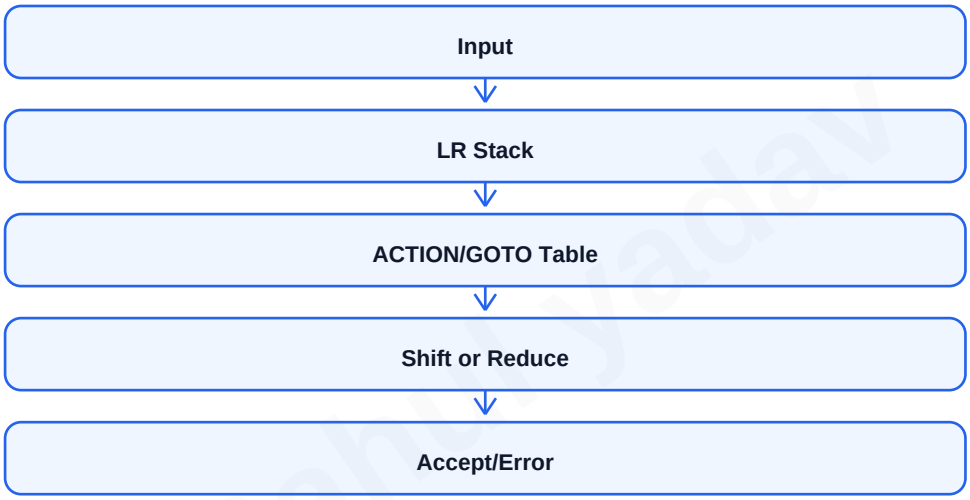
LR parsers are powerful bottom-up parsers. LR means scanning input from Left to right and producing Rightmost derivation in reverse. They use a stack and parsing table containing ACTION and GOTO entries.

Canonical LR(1) parser is most powerful among common LR methods but has large parsing tables. SLR is simpler but less powerful. LALR merges LR(1) states with same core and is widely used in practical parser generators.

YACC/Bison commonly generates LALR parsers. LR parsers can detect syntax errors as soon as possible in left-to-right scanning.

- LR parser uses stack of states and grammar symbols.
- ACTION table handles terminals: shift, reduce, accept or error.
- GOTO table handles non-terminals.
- SLR uses FOLLOW sets for reductions.
- LALR balances power and table size.

Parser	Power	Table Size	Use
SLR	Lowest among LR family	Small	Simple grammars
Canonical LR(1)	Highest	Large	Theory/complex grammars
LALR	Near LR(1)	Moderate	YACC/Bison practical parsers



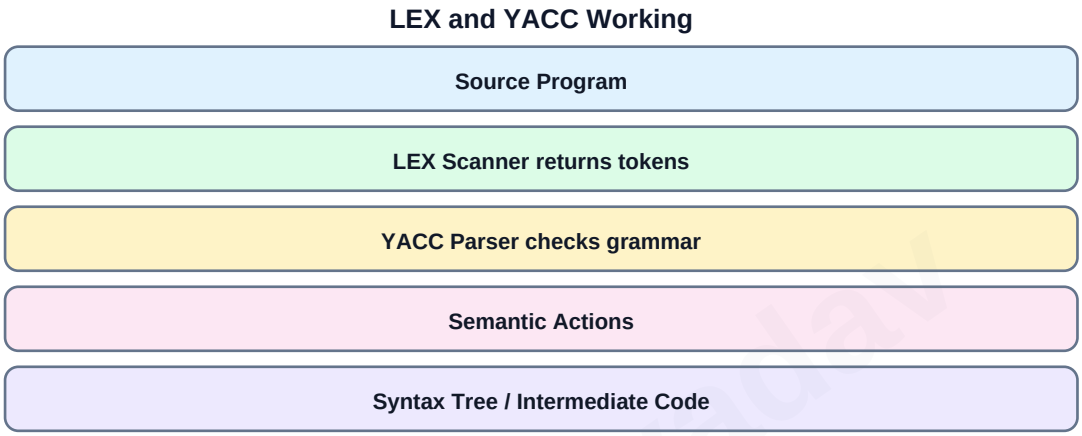
Parser Generation Tools

Parser generators automatically create parser code from grammar specification. The programmer writes grammar productions and semantic actions. The tool constructs parsing tables and generates parser program.

YACC and Bison are common parser generators. They are often used with LEX/Flex. Lexical analyzer returns tokens to parser, parser applies grammar rules and semantic actions build syntax tree or intermediate code.

Parser generators save development time and reduce errors compared to manually implementing complex LR parsers.

- Parser generator input is grammar specification.
- Output is parser source code.
- Semantic actions are executed when productions are reduced.
- LEX/Flex handles tokens; YACC/Bison handles grammar.
- Conflicts like shift-reduce are reported by tool.




```
expr : expr '+' term   { $$ = $1 + $3; }
     | term            { $$ = $1; }
     ;
```

Syntax Directed Definitions and Syntax Trees

Syntax directed definition attaches attributes and semantic rules to grammar productions. It is used to specify translation of programming language constructs, such as type checking, syntax tree construction and intermediate code generation.

An attribute can be synthesized or inherited. A synthesized attribute is computed from attributes of children nodes. An inherited attribute is computed from parent or sibling information.

A syntax tree is a compact tree representation of program structure. It removes unnecessary grammar details and keeps operators, operands and statements in meaningful hierarchy.

- SDD combines CFG with attributes and semantic rules.
- Synthesized attributes flow upward in parse tree.
- Inherited attributes flow downward or sideways.
- Syntax tree is more compact than parse tree.
- Semantic rules define how attributes are computed.

Attribute Type	Direction	Example
Synthesized	Child to parent	Expression value/type
Inherited	Parent/sibling to child	Declared type to id list
S-attributed SDD	Only synthesized	Easy for bottom-up
L-attributed SDD	Restricted inherited	Suitable for top-down

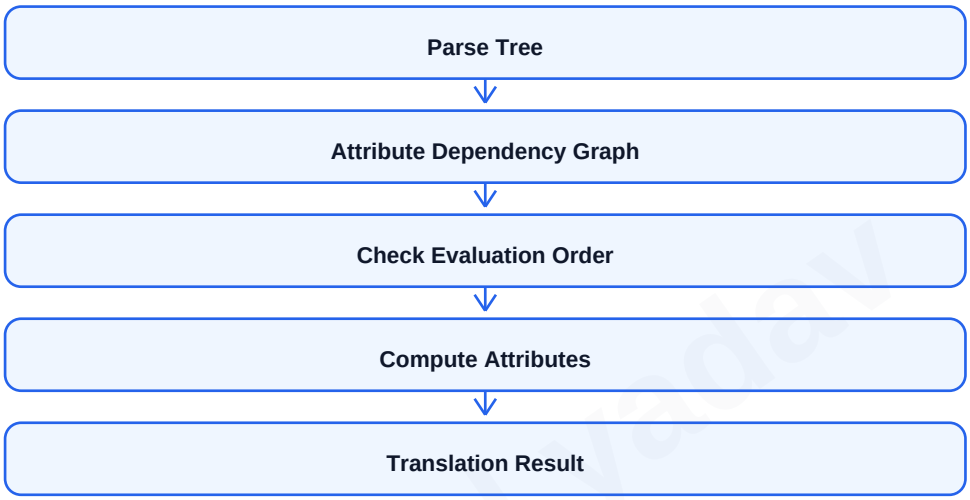
S-Attributed and L-Attributed Definitions

An S-attributed definition uses only synthesized attributes. Since attributes are computed from children to parent, S-attributed definitions are naturally evaluated during bottom-up parsing.

An L-attributed definition permits inherited attributes under certain restrictions. It can be evaluated in a single left-to-right traversal of parse tree and is suitable for top-down translation.

Syntax-directed translation schemes embed semantic actions inside grammar productions. Position of semantic action determines when it is executed.

- S-attributed definitions are simpler.
- L-attributed definitions allow controlled inherited information.
- Bottom-up evaluation suits synthesized attributes.
- Top-down translation can pass inherited attributes.
- Attribute dependency graph helps decide evaluation order.




```
Production: E -> E1 + T
Semantic rule: E.val = E1.val + T.val

Here E.val is synthesized from E1.val and T.val.
```

UNIT III: Type Checking and Type Systems

Type checking verifies that operations in a program are applied to compatible operands. It prevents errors such as adding integer to string without conversion, calling function with wrong parameter type or assigning incompatible value to variable.

A type system defines rules for assigning types to program constructs and checking their compatibility. Type checking can be static or dynamic. Static type checking happens at compile time, while dynamic type checking happens at run time.

Type checker uses information from symbol table, syntax tree and language type rules.

- Static checking catches errors before execution.
- Dynamic checking provides flexibility but errors appear at run time.
- Strong typing prevents unsafe operations.
- Type equivalence checks whether two types are same or compatible.
- Type conversion may be implicit or explicit.

Concept	Meaning
Type	Set of values and allowed operations
Type System	Rules for type assignment and compatibility
Type Checker	Compiler component that verifies type rules
Static Typing	Type checked at compile time
Dynamic Typing	Type checked at run time

Specification of Simple Type Checker

A simple type checker can be specified using syntax-directed definitions. Each expression node has a type attribute. For binary operators, the type checker verifies operand types and assigns result type.

For assignment statements, the type of left side and right side should be compatible. For relational expressions, operands must be comparable and result type is usually boolean. For function calls, actual parameter types must match formal parameter types.

When a type error is found, compiler reports clear message but may continue analysis to find more errors.

- id.type is obtained from symbol table.

- num.type is usually integer or real.

- E1 + E2 is valid if both operands are numeric.

- Assignment x = E is valid if E.type can be converted to x.type.

- Function call requires correct number and type of arguments.

Construct	Type Rule
E -> id	E.type = lookup(id).type
E -> num	E.type = integer
E -> E1 + E2	Both operands numeric, result numeric
S -> id = E	id.type compatible with E.type
E -> E1 relop E2	Operands comparable, result boolean

Type Equivalence, Conversion and Overloading

Type equivalence decides whether two types are considered same. Name equivalence means two types are same only if they have same declared name. Structural equivalence means two types are same if their internal structure is same.

Type conversion changes a value from one type to another. Widening conversion is usually safe, such as int to float. Narrowing conversion may lose information, such as float to int. Conversion may be implicit coercion or explicit cast.

Overloading means same operator or function name has multiple meanings depending on parameter types. The compiler resolves overloaded call by matching argument types.

- Name equivalence is strict and simple.
- Structural equivalence is flexible but more complex.
- Implicit conversion is inserted by compiler.
- Explicit conversion is written by programmer.
- Overload resolution selects best matching function/operator.

Topic	Meaning	Example
Name Equivalence	Same type name required	type A and type B differ
Structural Equivalence	Same structure required	records with same fields
Widening	Safe conversion	int to float
Narrowing	Possible loss	float to int
Overloading	Same name, different signatures	print(int), print(string)

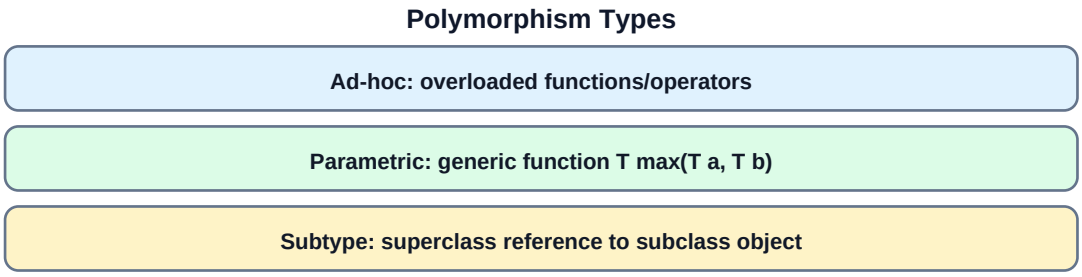
Polymorphic Functions and Operations

Polymorphism means one function or operation can work with values of different types. It improves reusability and abstraction. Polymorphism may be ad-hoc, parametric or subtype-based.

Ad-hoc polymorphism includes function overloading and operator overloading. Parametric polymorphism uses type parameters, such as generic functions. Subtype polymorphism allows object of subclass to be used where superclass is expected.

The compiler must check type correctness of polymorphic functions and choose appropriate implementation when needed.

- Ad-hoc polymorphism depends on overloading.
- Parametric polymorphism uses generic type variables.
- Subtype polymorphism is common in object-oriented languages.
- Type inference may determine generic type automatically.
- Polymorphism reduces duplicate code.




```
// Generic idea
T max(T a, T b) {
    return a > b ? a : b;
}
```

Runtime Environment and Storage Organization

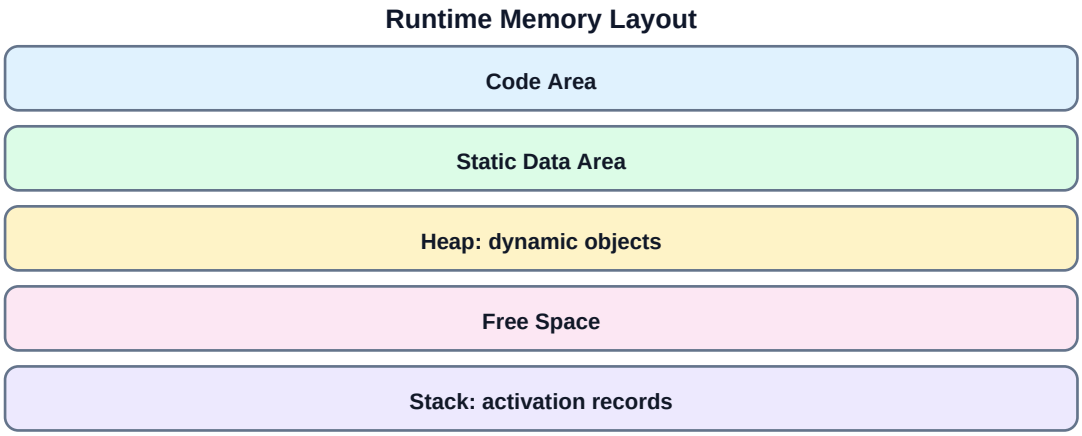
Runtime environment is the structure and support system needed to execute a program after compilation. It includes memory organization, activation records, parameter passing, dynamic storage allocation, symbol table information and runtime support routines.

Program memory is commonly divided into code area, static data area, stack and heap. Code area stores executable instructions. Static area stores global/static variables. Stack stores activation records for function calls. Heap stores dynamically allocated objects.

Compiler must generate code that correctly manages runtime storage for variables, temporaries, function calls and returns.

- Code area stores target instructions.

- Static area stores compile-time allocated data.
- Stack supports procedure calls and local variables.
- Heap supports dynamic allocation.
- Runtime organization affects performance and recursion support.



Memory Area	Stores	Lifetime
Code	Executable instructions	Whole program
Static	Global/static variables	Whole program
Stack	Local variables, return address	Function call duration
Heap	Dynamic objects	Until freed/garbage collected

Storage Allocation Strategies and Activation Records

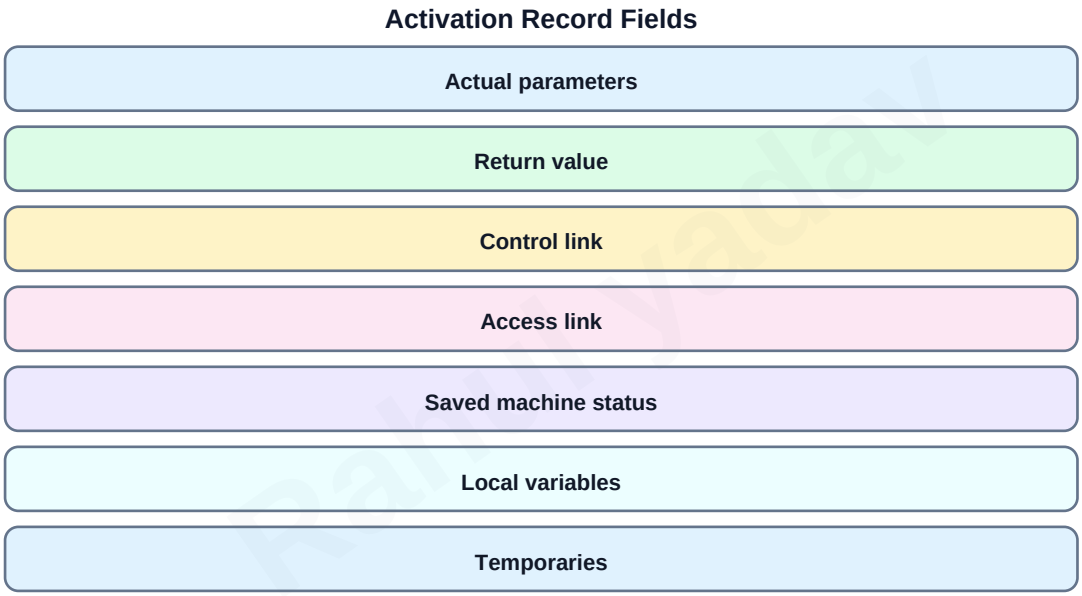
Storage allocation strategy decides where and when memory is allocated for program data. Static allocation assigns memory at compile time. Stack allocation assigns memory for procedure calls. Heap allocation manages data whose lifetime is not known at compile time.

An activation record, also called stack frame, stores information for one function call. It contains return address, parameters, control link, access link, local variables, temporaries and saved registers.

Stack allocation naturally supports recursion because every call gets a separate activation record.

- Static allocation is simple but does not support recursion well.
- Stack allocation is efficient for nested calls.
- Heap allocation is flexible but requires management.
- Activation record is pushed on function call and popped on return.
- Control link points to caller activation record.

Strategy	Allocation Time	Use
Static	Compile/load time	Global variables
Stack	Procedure call time	Local variables and parameters
Heap	Run time on request	Objects, dynamic data structures



Parameter Passing, Symbol Table and Error Recovery

Parameter passing defines how values are transferred between caller and called procedure. Common methods are call by value, call by reference, call by result, call by value-result and call by name.

The symbol table stores information about identifiers such as name, type, scope, size, memory location, parameter list and return type. It is used by lexical analyzer, semantic analyzer, code generator and optimizer.

Error detection and recovery allow compiler to report errors and continue compilation. Ad-hoc methods are simple manual strategies, while systematic methods use formal recovery techniques such as panic mode, phrase-level recovery and error productions.

- Call by value passes copy of value.
- Call by reference passes address, so changes affect actual argument.
- Symbol table supports scope handling.
- Panic mode skips input until synchronizing token.
- Good error messages should include location and reason.

Parameter Method	Meaning
Call by Value	Formal receives copy
Call by Reference	Formal receives address/reference
Call by Result	Formal copied back on return
Value-Result	Copy in and copy out
Call by Name	Argument expression re-evaluated when used

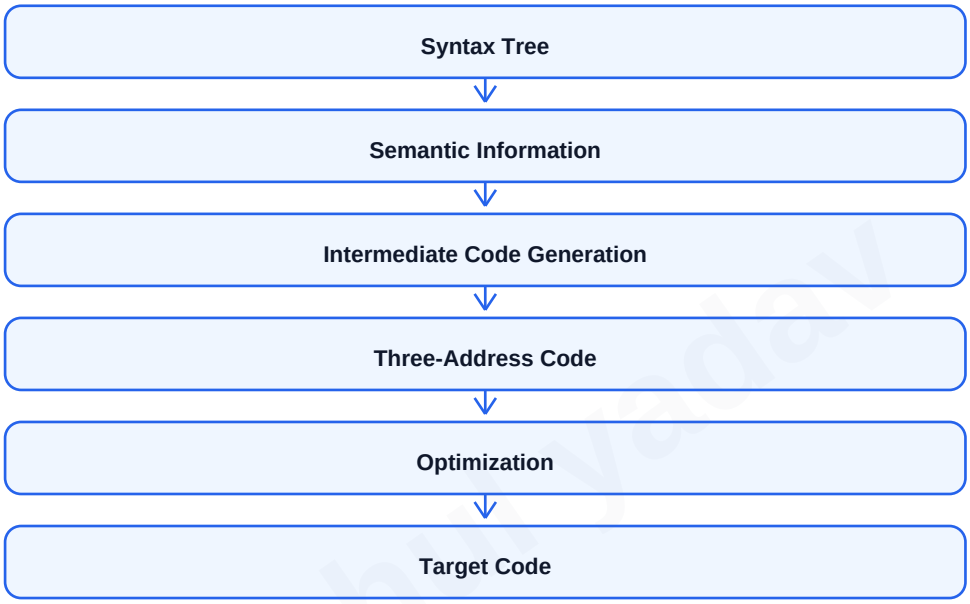
UNIT IV: Intermediate Code Generation

Intermediate code generation converts source program into an intermediate representation that is easier to optimize and translate to machine code. Intermediate code separates front end and back end, making compiler more portable.

Three-address code is a common intermediate representation. Each instruction has at most three addresses: two operands and one result. It is easy to generate and easy to optimize.

Intermediate code is generated for declarations, assignment statements, boolean expressions, case statements and procedure calls.

- Intermediate code should be easy to generate.
- It should be machine independent.
- It should be easy to optimize.
- Three-address code uses temporary variables.
- IR helps retarget compiler to multiple machines.



IR Form	Meaning
Three-address code	Sequence of simple instructions
Syntax tree	Tree representation of program
DAG	Compact expression representation
Postfix notation	Operator after operands
Quadruples	op, arg1, arg2, result representation

Three-Address Code for Assignments and Declarations

Assignment expressions are converted into a sequence of simple operations using temporary variables. Complex expressions are broken into smaller operations that are easier to optimize and translate.

Declarations allocate storage and enter identifier information into symbol table. Type and size information are used to compute memory offsets for variables.

Three-address code can be represented as quadruples, triples or indirect triples. Quadruple has four fields: operator, argument1, argument2 and result.

- Each TAC instruction contains at most one operator.
- Temporary variables store intermediate results.
- Declarations update symbol table and offsets.
- Quadruples are easy to rearrange during optimization.
- Triples refer to instruction positions instead of explicit temporaries.

Representation	Fields	Example
Quadruple	op, arg1, arg2, result	(+, a, b, t1)
Triple	op, arg1, arg2	0: (+, a, b)
Indirect Triple	Pointer table + triples	Allows code movement

```
x = a + b * c

t1 = b * c
t2 = a + t1
x  = t2
```

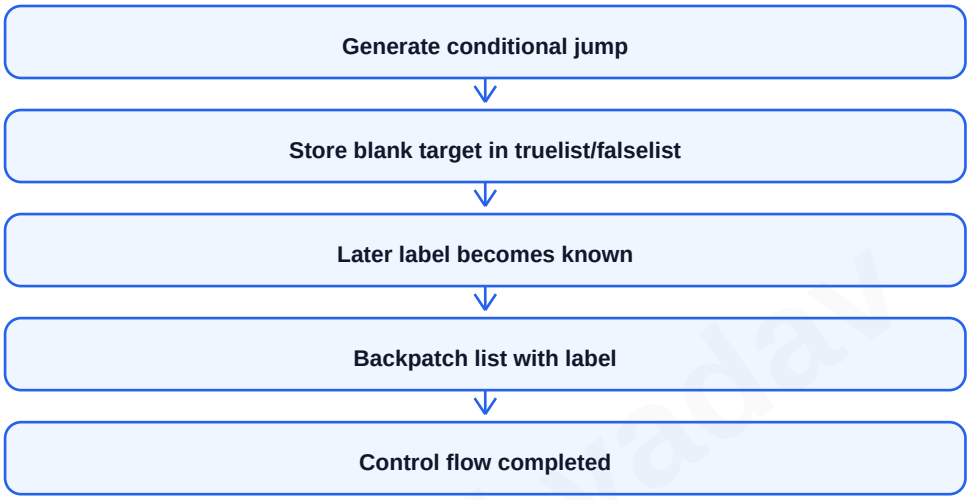
Boolean Expressions and Backpatching

Boolean expressions are used in conditional statements and loops. Intermediate code for boolean expressions often uses jumps instead of computing true/false values explicitly.

Backpatching is a technique used to fill target labels of jumps later when those labels become known. It is useful in one-pass code generation for boolean expressions and control-flow statements.

The compiler maintains lists such as truelist, falselist and nextlist. These lists contain incomplete jump instructions that are patched later.

- truelist contains jumps to be taken when expression is true.
- falselist contains jumps to be taken when expression is false.
- nextlist contains jumps to next statement.
- makelist creates a list with one instruction index.
- merge combines two lists, and backpatch fills target label.




```
if a < b goto _
goto _

// Later:
backpatch(E.truelist, Ltrue)
backpatch(E.falselist, Lfalse)
```

Case Statements and Procedure Calls

Case or switch statements can be translated using a sequence of conditional jumps, jump table or binary search depending on number and density of case values. Jump tables are efficient when case values are dense.

Procedure call code generation must handle parameter passing, activation record setup, control transfer and return value. Intermediate code often uses param, call and return instructions.

The compiler must preserve calling conventions so caller and callee agree about stack layout, registers and return values.

- Linear search is simple for small number of cases.
- Jump table gives O(1) branching for dense case labels.
- Procedure call pushes parameters and return address.
- Return statement transfers value back to caller.
- Calling convention defines register/stack usage.

Construct	Possible Translation
case/switch	Conditional jumps or jump table
Procedure call	param x; call p, n
Return	return value
Function result	Stored in register or caller-provided location
Parameters	Stack/register according to convention

```
param a
param b
t1 = call sum, 2
x = t1
```

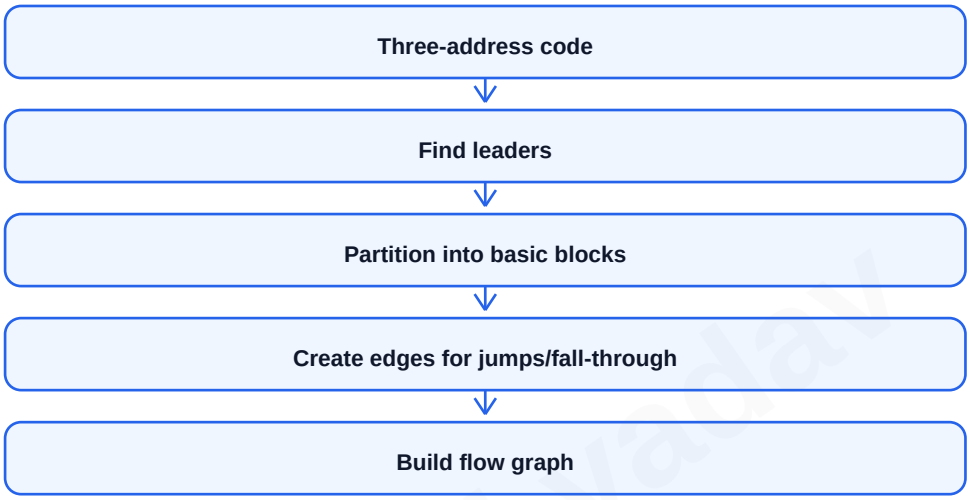
Code Generation Issues and Basic Blocks

Code generation translates intermediate code into target machine instructions. Important issues include instruction selection, register allocation, order of evaluation, handling memory addresses, use of addressing modes and preserving correctness.

A basic block is a sequence of consecutive statements with single entry and single exit. Control enters at first statement and leaves at last statement without branching inside. Basic blocks are used for local optimization.

A flow graph represents control flow among basic blocks. Nodes are basic blocks and edges represent possible transfer of control.

- Leaders are first statements of basic blocks.
- First statement is always a leader.
- Target of a jump is a leader.
- Statement after a jump is a leader.
- Flow graph helps data-flow analysis and optimization.



Issue	Meaning
Instruction Selection	Choose target instructions
Register Allocation	Assign variables to registers
Addressing Modes	Use efficient memory access forms
Evaluation Order	Reduce registers and instructions
Control Flow	Generate correct jumps/labels

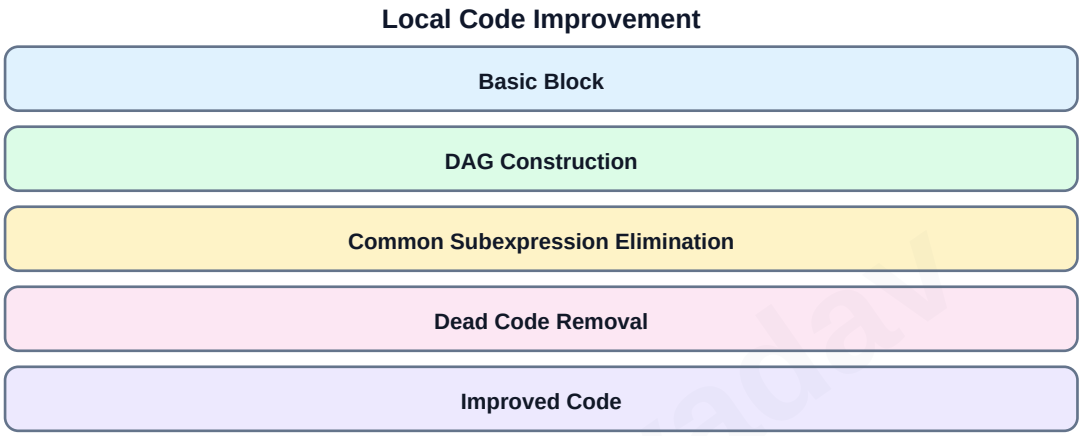
Register Allocation, DAG and Peephole Optimization

Register allocation decides which values should be kept in CPU registers. Since registers are limited, compiler must choose carefully. Register assignment selects exact register for each value.

A DAG representation of a basic block captures common subexpressions and dependencies between computations. It helps eliminate redundant calculations and remove dead assignments within a basic block.

Peephole optimization examines a small window of target instructions and replaces inefficient instruction sequences with better ones.

- Good register allocation reduces memory access.
- Graph coloring is a common register allocation approach.
- DAG detects common subexpressions inside a basic block.
- Peephole optimization is local and machine dependent.
- Examples include removing redundant loads and algebraic simplification.



Optimization	Example
Redundant load/store removal	load R1,a; load R1,a -> one load
Algebraic simplification	x * 1 -> x
Strength reduction	x * 2 -> x + x or shift
Unreachable code removal	Remove code after unconditional jump
Common subexpression	Reuse a+b instead of recomputing

UNIT V: Introduction to Code Optimization

Code optimization improves intermediate or target code so that the program runs faster, uses less memory or consumes fewer resources while preserving original meaning. Optimization should never change program output.

Optimization may be machine independent or machine dependent. Machine-independent optimization works on intermediate code, while machine-dependent optimization uses target machine features such as registers and addressing modes.

Optimization can be local, global or loop-based. Local optimization works within a basic block. Global optimization works across basic blocks. Loop optimization focuses on loops because programs spend much time inside loops.

- Optimization must preserve semantics.
- It may improve time, space or power consumption.
- Local optimization is easier and cheaper.
- Global optimization uses flow graph information.
- Loop optimization has high impact because loops execute repeatedly.

Type	Scope	Example
Local	Single basic block	Common subexpression in block
Global	Multiple basic blocks	Available expression analysis
Loop	Inside loops	Loop-invariant code motion
Machine-dependent	Target instructions	Peephole/register optimization
Machine-independent	Intermediate code	Constant folding

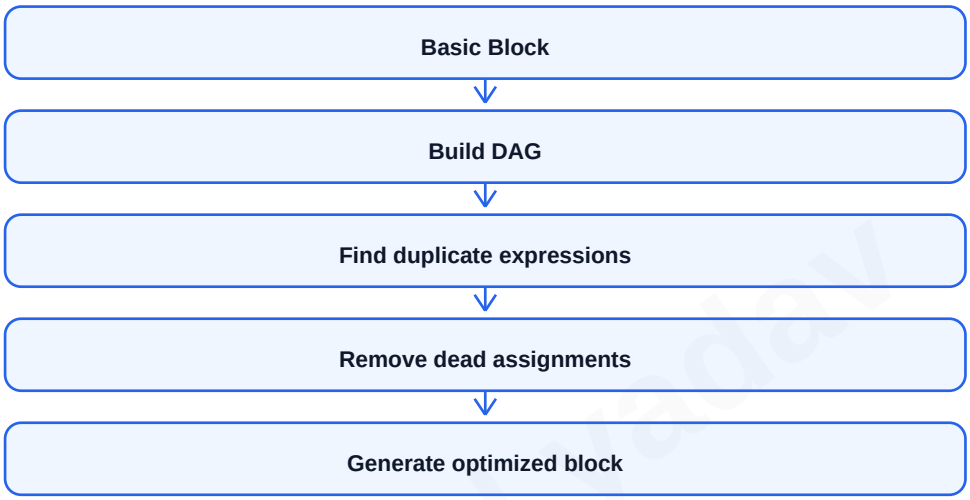
Sources of Optimization in Basic Blocks

Basic block optimization uses local information to improve code. Since control does not branch inside a basic block, dependencies are easier to analyze. DAG is often used to identify repeated expressions and dead assignments.

Common local optimizations include common subexpression elimination, constant folding, algebraic simplification, copy propagation and dead code elimination.

Constant folding evaluates constant expressions at compile time. Copy propagation replaces a copied variable by original variable. Dead code elimination removes statements whose results are never used.

- Common subexpression: reuse previously computed expression.
- Constant folding: 3 * 4 becomes 12.
- Algebraic identities: x + 0 becomes x.
- Copy propagation: after x = y, replace x by y when safe.
- Dead assignment: remove x = value if x is never used.




```
Before:
t1 = a + b
t2 = a + b
x = t2

After:
t1 = a + b
x = t1
```

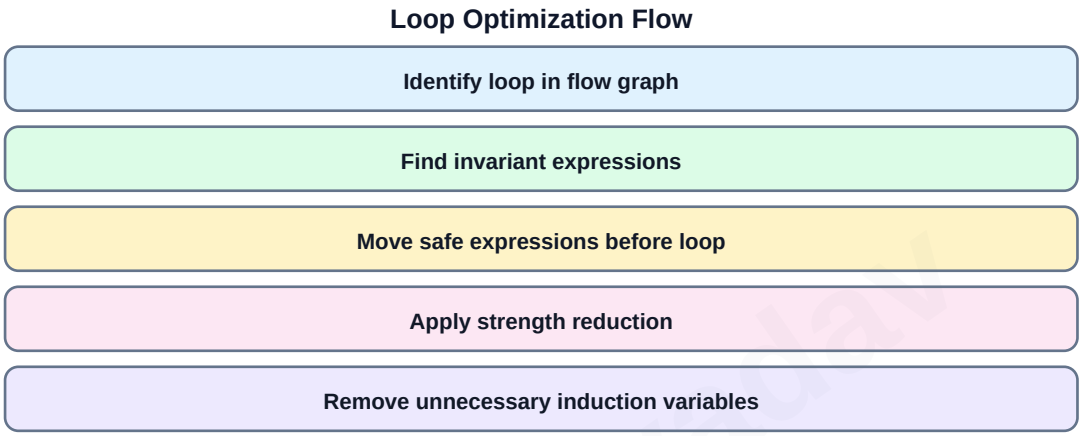
Loops in Flow Graphs and Loop Optimization

A loop in a flow graph is a set of nodes with a back edge. A back edge goes from a node to one of its dominators. Loop optimization is important because even small improvements inside loops can greatly improve total execution time.

Common loop optimizations are loop-invariant code motion, induction variable elimination, strength reduction, loop unrolling and reducing frequency of expensive operations.

Loop-invariant code motion moves computations outside loop if their value does not change during loop execution.

- Back edge identifies natural loop.
- A dominator is a node that must be passed to reach another node.
- Loop-invariant computations can move before loop.
- Strength reduction replaces expensive operation with cheaper one.
- Induction variables change by constant amount each iteration.




```
Before:
while (i < n) {
    x = y + z;   // y and z do not change
    a[i] = x + i;
}

After:
x = y + z;
while (i < n) {
    a[i] = x + i;
}
```

Dead Code Elimination and Code Improving Transformations

Dead code is code that does not affect program output. It may be unreachable code or computations whose results are never used. Removing dead code reduces program size and execution time.

Code improving transformations modify code to improve efficiency without changing meaning. They include constant propagation, common subexpression elimination, copy propagation, strength reduction and algebraic simplification.

Optimization must be conservative. A transformation should be applied only when compiler can prove that program behavior remains same.

- Unreachable code occurs after unconditional jump or return.
- Dead assignment assigns value that is never used.
- Constant propagation replaces variable with known constant.
- Strength reduction replaces costly operations.
- Optimization should consider side effects such as function calls and volatile variables.

Transformation	Before	After
Constant Folding	x = 2 * 5	x = 10
Algebraic Simplification	x = y + 0	x = y
Strength Reduction	x = i * 4	x = i << 2
Copy Propagation	x = y; z = x + 1	z = y + 1
Dead Code	x = 5; x never used	remove statement

Global Data Flow Analysis

Global data flow analysis collects information about how data values move through a program's control flow graph. It is used for global optimizations such as reaching definitions, live variable analysis, available expressions and constant propagation.

A data flow problem is usually expressed using IN and OUT sets for each basic block along with GEN and KILL sets. Iterative algorithms compute these sets until no further changes occur.

Reaching definitions tells which assignments may reach a program point. Live variable analysis tells whether a variable value may be used in future.

- Data flow analysis works on control flow graph.
- GEN set contains information generated by a block.
- KILL set contains information invalidated by a block.
- Forward analysis flows from entry to exit.
- Backward analysis flows from exit to entry.

Analysis	Direction	Use
Reaching Definitions	Forward	Find assignments reaching a point
Available Expressions	Forward	Common subexpression elimination
Live Variables	Backward	Register allocation/dead code
Very Busy Expressions	Backward	Code motion

```
Forward example:
OUT[B] = GEN[B] union (IN[B] - KILL[B])

Backward example:
IN[B] = USE[B] union (OUT[B] - DEF[B])
```

Symbolic Debugging of Optimized Code

Symbolic debugging allows programmers to debug using source-level names and line numbers instead of machine addresses. Optimization makes debugging harder because code may be reordered, variables may be removed or stored in registers and statements may be combined.

In optimized code, a source statement may correspond to multiple machine instructions or no instruction at all. A variable may not have a fixed memory location throughout execution.

Compilers generate debug information that maps target code back to source code. However, debugging optimized code can still show surprising behavior, so sometimes optimization is disabled during debugging.

- Optimization can reorder instructions.
- Dead variables may be removed.
- Variables may live in registers instead of memory.
- Inlining can remove function boundaries.
- Debug metadata maps machine code to source names and lines.

Optimization Effect	Debugging Problem
Code motion	Execution order differs from source
Dead code removal	Breakpoint may not be hit
Register allocation	Variable memory address changes
Inlining	Function call frame may disappear
Common subexpression	Expression not recomputed at source line

Important Compiler Design Programs and Examples

Compiler Design practical questions often ask for token recognition, FIRST/FOLLOW computation, recursive descent parser, intermediate code generation and simple code optimization. The main idea is to show clear input, processing and output.

For token recognition, read source string and classify lexemes. For parser programs, implement grammar functions or use stack/table. For intermediate code, generate temporaries. For optimization, apply simple transformations such as constant folding and dead code removal.

- Lexical analyzer: identify keywords, identifiers, operators and constants.
- Recursive descent parser: implement one function for each non-terminal.
- FIRST/FOLLOW: repeatedly apply grammar rules until sets become stable.
- Three-address code: break complex expression into temporary assignments.
- Basic block: find leaders and partition TAC.
- DAG: remove common subexpressions inside block.

Practical Topic	Input	Output
LEX token recognizer	Source program	Token list
Recursive descent parser	Expression/string	Accepted or rejected
FIRST/FOLLOW	Grammar	Sets
Intermediate code	Expression/statement	TAC
Code optimization	TAC/basic block	Optimized code

Quick Revision: High-Scoring Long Answers

For compiler phases, draw the phase diagram and write input-output of each phase. For lexical analysis, explain token, lexeme, pattern, regular expression, DFA and input buffering. For parsing, compare top-down and bottom-up parsing with examples.

For syntax directed translation, define attributes and explain synthesized/inherited attributes. For runtime environment, draw memory layout and activation record. For code generation, explain TAC, basic blocks, flow graph and register allocation.

For optimization, write meaning, need, levels of optimization and examples. Use tables for differences and small code blocks for transformations.

- Compiler phases are the most common long question.
- Lexical analyzer generator LEX is important for tool-based questions.
- FIRST and FOLLOW are important for predictive parsing.
- SLR, LALR and LR comparison is exam-friendly.
- Activation record diagram is important in runtime environment.
- Backpatching is important for boolean expressions.
- Loop optimization and data flow analysis are important in Unit V.

Question	Must Include
Phases of compiler	Diagram + each phase output
Lexical analysis	Token/lexeme/pattern + DFA + buffering
Predictive parsing	FIRST/FOLLOW + LL(1) table
LR parsers	ACTION/GOTO + SLR/LALR/LR comparison
Runtime environment	Memory layout + activation record
Optimization	Types + examples + data flow